理解 FlashAttention

Understanding FlashAttention

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
Memory Traffic	内存流量	
FlashAttention	FlashAttention（方法名保留）	
Online Softmax	在线 Softmax	
IO Complexity	输入输出复杂度	
Attention	注意力机制	Attention
Exact Attention	精确注意力	
Floating-point	浮点	
Rounding	舍入	
Low-rank	低秩	
Softmax	Softmax 归一化	Softmax
Core	核心	
Scaled Dot-product Attention	缩放点积注意力	SDPA
Dense Attention	稠密注意力	
Query	查询	Q
GPU Memory Hierarchy	GPU 内存层次结构	
High-bandwidth Memory	高带宽内存	HBM
FLOP Count	浮点运算次数	
Values	值	
Attention Head	注意力头	
Self-attention	自注意力	
Score Matrix	分数矩阵	
Additive Mask	加性掩码	
GPU Generation	GPU 代际	
Shared Memory	共享内存	
Registers	寄存器	
Blockwise Merge	分块合并	
Backward Pass	反向传播	

英文原词	中文译名	缩写／符号
Forward Pass	前向传播	
Queries	查询	
Tiling	分块	
FLOPs	浮点运算次数	
Keys	键	
MHA	多头注意力	MHA
MQA	多查询注意力	MQA
GQA	分组查询注意力	GQA
Mental Model	理解框架	
Generation	生成	
Dot-product Attention	点积注意力	
Work Partitioning	工作划分	
Linear Attention	线性注意力	
Bitwise Identity	逐位一致性	
Local Attention	局部注意力	
PagedAttention	分页注意力	
Data Movement	数据移动	
Performance	性能	
Precision	精度	
Training	训练	
Scaling	缩放	
Prefill	预填充	
Dropout	随机失活	
Matmul	矩阵乘法	
Decode	解码	
GEMM	通用矩阵乘法	GEMM
Row-wise Softmax	逐行 Softmax	
Matrix	矩阵	
Kernel	内核	

英文原词	中文译名	缩写／符号
Model	模型	
RoPE	旋转位置嵌入	RoPE
Matrix Multiplication	矩阵乘法	
Recomputation	重计算	
Throughput	吞吐量	
Operator	算子	
Device Memory	设备内存	
Tensor	张量	
Models	模型	
Resident	驻留	
Head Dimension	头维度	
Memory-bound	内存受限	
Dtype	数据类型	
Mask	掩码	
Heads	头	
Tokens	词元	
Occupancy	占用率	
Efficient Attention	高效注意力	
Normalization	归一化	
Memory Hierarchy	内存层次结构	
Accumulator	累加器	
Vector	向量	
Accumulation	累加	
Warp	线程束	
Batching	批处理	
Floating Point	浮点	
Memory Footprint	内存占用	
Transformer	Transformer 模型	Transformer
Activation	激活	

英文原词	中文译名	缩写／符号
Serving	推理服务	
Causal Self-attention	因果自注意力	
Gradient	梯度	
Synchronization	同步	
Latency	延迟	
Kernel Launch	内核启动	
Tensor Cores	张量核心	
Tensor-core Throughput	张量核心吞吐量	
Value Heads	值头	
Query Heads	查询头	
KV Heads	键值头	
Query Head	查询头	
Inference	推理	
Causal Attention	因果注意力	
KV Cache	键值缓存	KV Cache
Thread Block	线程块	
Warps	线程束	
Benchmark	基准测试	
Pipeline	流水线	
Batch Size	批大小	
FLOP	浮点运算	
Accuracy	准确性	
Quantization	量化	
Tensor Memory Accelerator	张量内存加速器	TMA
Quantization Error	量化误差	
Baseline	基线	
Memory Bandwidth	内存带宽	
MMA	矩阵乘加	MMA
Evidence	证据	

英文原词	中文译名	缩写／符号
GPU Utilization	GPU 利用率	
Fused Kernel	融合内核	
Fragmentation	碎片化	
Local Window	局部窗口	
Token	词元	
Cache Capacity	缓存容量	
Profiling	性能剖析	
Boolean Mask	布尔掩码	
Large Language Model	大语言模型	LLM

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

理解 FlashAttention · 第 9 页

如何阅读本手册 · 第 10 页

内容 · 第 11 页

内容续 · 第 12 页

1 FlashAttention实际优化了什么 · 第 13 页

2 注意力机制（Attention）方程不是实现 · 第 14 页

3 GPU 内存层次结构（GPU Memory Hierarchy）以及 IO 为何重要 · 第 15 页

4 为什么物化（公式）和（公式）很贵 · 第 16 页

5 密集算术仍然是二次的 · 第 17 页

6 内存高效的精确注意力（Exact Attention）早于 FlashAttention · 第 18 页

7 平铺查询（Queries）、键和值 · 第 19 页

8 Softmax 是流媒体的难点 · 第 20 页

9 从第一原理开始的在线softmax · 第 21 页

10 分块合并（Blockwise Merge）递归 · 第 22 页

11 一个完整的数值例子 · 第 23 页

12 FlashAttention 前传 · 第 24 页

13 为什么密集的FlashAttention是准确的 · 第 26 页

14 IO 复杂性：定理的实际含义 · 第 27 页

15 内存复杂度：线性辅助状态，非线性计算 · 第 28 页

16 因果掩蔽和瓦片跳跃 · 第 29 页

17 向后传递：重新计算而不是存储 · 第 30 页

18 为什么更多的 FLOP 仍然可以更快 · 第 31 页

19 MHA、MQA 和 GQA 兼容性 · 第 32 页

20 可变长度、局部注意力（Local Attention）和 dropout · 第 33 页

21 FlashAttention-2：发生了什么变化 · 第 34 页

22 FA2 跨序列图块的并行性 · 第 35 页

23 FA2 工作分区和非 matmul FLOP · 第 36 页

24 FlashAttention-3：Hopper一代 · 第 37 页

25 FA3 异步：重叠数据移动（Data Movement）、GEMM 和 softmax · 第 38 页

26 FA3 FP8：不假装精确的性能（Performance）是免费的 · 第 39 页

27 FlashAttention-4：布莱克威尔一代 · 第 40 页

28 FA4 和非对称硬件缩放（Scaling） · 第 41 页

29 FA4实施及现状 · 第 42 页

30 FlashAttention-1 到 -4 比较 · 第 43 页

31 PyTorch 缩放点积今日关注 · 第 44 页
32 精确性不是按位恒等 · 第 45 页
33 FlashAttention 与 PagedAttention · 第 46 页
34 FlashAttention 与稀疏和线性注意力（Linear Attention） · 第 47 页
35 训练（Training）、预填充（Prefill）和解码（Decode）是不同的机制 · 第 48 页
36 种常见的实施错误 · 第 49 页
37 常见误解和实用心智模式 · 第 50 页
参考文献和进一步阅读 · 第 51 页
参考图：针对哪个问题阅读什么内容 · 第 53 页

理解 FlashAttention

对应原稿第 1 页

了解 FlashAttention

IO 感知注意力机制（Attention）如何在无需近似注意力机制的情况下使 Transformer 更快

> 关于精确平铺注意力机制（Attention）的技术手册：GPU 内存流量（Memory Traffic）、在线 softmax、前向和反向传播、IO 复杂性、从 FlashAttention-1 到 FlashAttention-4 的演变以及当前的框架行为。

如何阅读本手册

对应原稿第 2 页

当三个不同的问题分开时，FlashAttention 最容易理解：

1. 正在计算什么数学函数？对于稠密注意力（Dense Attention），目标仍然是普通的缩放点积注意力（Scaled Dot-product Attention）。

2. 该函数需要多少算术运算量？密集的全对查询（Query）键评分在序列长度上保持二次方。

3. 实现如何通过 GPU 内存层次结构（GPU Memory Hierarchy）移动数据？这就是 FlashAttention 显着改变算法执行的地方。[4]

核心教训是实际运行速度不仅仅由 FLOP 计数决定。算法基本上可以执行相同的数学工作，有时甚至可以重新计算中间值，但运行速度更快，因为它在高带宽内存（High-bandwidth Memory）之间移入和移出的数据要少得多。

> 核心区别。 Dense FlashAttention 是一种精确注意力（Exact Attention）算法：它不会用低秩（Low-rank）、稀疏、核化或近似公式取代 softmax 注意力机制（Attention）。“精确”指的是数学注意力机制计算。由于操作被重排序，浮点（Floating-point）内核仍然可能因较小的舍入（Rounding）效应而有所不同。[4,10,13]

符号

对于一个注意力头（Attention Head），让

$$Q \in \mathbb{R}^{N_q \times d}, \qquad K \in \mathbb{R}^{N_k \times d}, \qquad V \in \mathbb{R}^{N_k \times d_v}.$$

对于 self-attention，通常为 $N_q = N_k = N$.的缩放分数矩阵（Score Matrix）为

$$S = \frac{QK^\top}{\sqrt{d}} + B,$$

其中 B表示可选的附加掩模或偏置，并且

$$O = PV.$$

$P = \mathrm{softmax}_{\mathrm{row}}(S)$,

我们使用HBM为大型片外高带宽GPU存储器和片上存储器作为广泛的教学

更小、更快的存储的术语，例如寄存器（Registers）和共享内存（Shared Memory）/SRAM。确切的硬件详细信息因 GPU 代而异。

内容

对应原稿第 3 页

1 FlashAttention 实际优化的是什么 5 2 注意力机制（Attention）方程不是实现 6 3 GPU 内存层次结构（GPU Memory Hierarchy）以及为什么 IO 很重要 7 4 为什么物化 S和 P成本高昂 8 5 密集算术仍然是二次方 9 6 内存高效的精确注意力（Exact Attention）早于 FlashAttention 10 7 平铺查询（Queries）、键和值 11 8 Softmax 是流处理中最困难的部分 12 9 从第一原理开始的在线 softmax 13 10 分块合并（Blockwise Merge）递归 14 11 一个完整的数值示例 15 12 FlashAttention 前向传播（Forward Pass） 16 13 为什么密集 FlashAttention 是精确的 17 14 IO 复杂性：定理实际上说了什么 18 15 内存复杂性：线性辅助状态，而不是线性辅助状态线性计算 19 16 因果掩码和切片跳过 20 17 向后传递：重新计算而不是存储 21 18 为什么更多的 FLOP 仍然可以更快 22 19 MHA、MQA 和 GQA 兼容性 23

内容续

对应原稿第 4 页

20 可变长度、局部注意力（Local Attention）和 dropout 24 21 FlashAttention-2：发生了什么变化 25 22 FA2 跨序列图块的并行性 26 23 FA2 工作分区和非 matmul FLOP 27 24 FlashAttention-3：Hopper 生成（Generation） 28 25 FA3 异步：重叠数据移动（Data Movement）、GEMM 和 softmax 29 26 FA3 FP8：无需伪装精度（Precision）的性能（Performance）是免费的 30 27 FlashAttention-4：Blackwell 一代 31 28 FA4 和非对称硬件缩放（Scaling） 32 29 FA4 实现和当前状态 33 30 FlashAttention-1 到 -4 比较 34 31 PyTorch 缩放点产品今天的关注35 32 精确性不是按位恒等 36 33 FlashAttention 与 PagedAttention 37 34 FlashAttention 与稀疏和线性注意力（Linear Attention） 38 35 训练（Training）、预填充（Prefill）和解码（Decode）是不同的机制 39 36 常见的实现错误 40 37 常见的误解和实用的理解框架（Mental Model） 41 参考文献和进一步阅读 42

FlashAttention 系列随 GPU 硬件一起发展。第 1-20 章解释了持久的算法思想。第 21-31 章隔离了特定于代际和特定于框架的细节，以便核心理解框架（Mental Model）不依赖于一个 CUDA 版本或一个 GPU 代际（GPU Generation）。

1 FlashAttention实际优化了什么

对应原稿第 5 页

从普通的注意力机制（Attention）函数开始：

$$\varsigma\left(\frac{QK^\top}{\sqrt{d}}\right.$$

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}} + B\right)V.$$

教科书的实现通常使它看起来像三个大型运算：形成分数矩阵（Matrix）S,，应用行式softmax来获得 P,，然后将 P乘以 V.从数学上来说这是很好的。然而，在 GPU 上，将巨大的中间矩阵写入 HBM 并将其读回的成本可能比等式所示的要高得多。

FlashAttention 的核心贡献是使算法具有 IO 感知能力。它将计算划分为适合快速片上内存的图块，流式传输 K和 V,块，并保持足够的逐行 softmax 状态以产生精确的输出，而无需在 HBM 中实现完整的 $N \times N$ 注意力机制（Attention）矩阵（Matrix）。 [4]

> 发生了什么变化：执行计划、内存流量（Memory Traffic）和存储的中间体。不变的是：正在评估密集的缩放点积注意力（Dot-product Attention）函数。

这种区别就是为什么“FlashAttention 是一种更快的注意力机制（Attention）”可能会产生误导。最好将其视为一种算法和内核（Kernel）系列，用于有效评估加速器上的注意力机制。模型（Model）可以使用因果掩码、RoPE、MQA/GQA 或其他注意功能，并且仍然在下面使用 FlashAttention 实现。

原始论文将这种方法与通过改变数学问题来减少算术的近似注意方法进行了对比。 Dense FlashAttention 不会进行该交易。同一篇论文还介绍了块稀疏扩展，但稀疏扩展是一种不同的情况，因为省略块会改变计算交互的方式。 [4]

2 注意力机制（Attention）方程不是实现

对应原稿第 6 页

两种实现可以计算相同的数学函数，同时移动不同数量的数据。

一个简单的物化实现在概念上可以做到：

$$S \leftarrow QK^\top/\sqrt{d}, \qquad P \leftarrow \mathrm{softmax}(S), \qquad O \leftarrow PV.$$

该演示隐藏了张量所在的位置。如果在第一次矩阵乘法（Matrix Multiplication）后将 S写入HBM，读取softmax，写回 P，然后再次读取 P进行 PV乘法，则GPU花费大量时间移动 N_2对象记忆。

FlashAttention 重新组织相同的依赖关系。 Q, K,和 V块被带到计算单元附近，分数图块在本地生成和消耗，并且只有紧凑的行统计数据加上输出需要跨图块持久保存。 [4, 5]

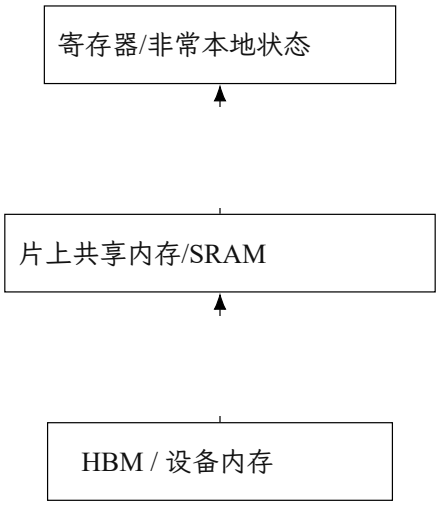
> 算法课。计算图不是内存调度。在纸上写 $P = \mathrm{softmax}(QK^\top)$不需要实现将所有 $QK^\top$或 P一次存储在片外存储器中。

这个想法超出了人们的注意力机制（Attention）。融合内核、平铺、重新计算和运算符调度通常会用少量的额外算术来换取更少的大型中间体移动。在现代加速器上，这可能是正确的选择，因为矩阵（Matrix）乘法吞吐量（Throughput）的增长速度比内存和执行层次结构的许多其他部分快得多。 FlashAttention-3 和 -4 使硬件依赖性变得越来越明确。[6, 7]

3 GPU 内存层次结构（GPU Memory Hierarchy）以及 IO 为何重要

对应原稿第 7 页

GPU 暴露的是一种层次结构，而不是一个同样快速的内存的统一池。确切的名称和功能因架构而异，但一个有用的思维模型是：



HBM 很大，但数据必须传输到计算单元。片上存储要小得多，但重复使用要便宜得多。原始的 FlashAttention 分析使用 HBM 和 SRAM 模拟这种不对称性，并显式优化它们之间的传输数量。 [4]

平铺的目的不仅仅是“让张量更小”。这是为了在块驻留（Resident）在芯片上时增加重用。在写回其部分 softmax/输出状态之前，Q块可以与多个 K/V块进行交互。相反， K/V块可以根据所选计划通过查询（Query）块进行流式传输。

> 当数据的放置和移动成为算法的一部分时，内核（Kernel）就会具有 IO 感知能力，而不是事后才考虑一系列单独启动的张量（Tensor）操作。

不要把这变成一个普遍的口号，即注意力机制（Attention）总是“受记忆限制”。瓶颈取决于序列长度、头维度（Head Dimension）、数据类型（Dtype）、掩码（Mask）模式、GPU 生成、前向与后向以及正在运行的内核（Kernel）。 FA3 和 FA4 存在的部分原因是，随着硬件的变化，主要瓶颈也发生了变化。[6, 7]

4 为什么物化 S和 P很贵

对应原稿第 8 页

二次中间体很快就变得具体了。假设一个批次包含一个序列，有 32 个注意力机制（Attention）头，序列长度 N= 8192，以及一个两字节的数据类型（Dtype），例如 FP16 或 BF16。具有形状的一个稠密张量（Tensor）

$$[1, 32, 8192, 8192]$$

包含32×8192^2元素。每个元素两个字节，即

$$32 \times 8192^2 \times 2 = 4,294,967,296 \text{ bytes} = 4 \text{ GiB}.$$

对于本例来说，这是一个满分或类概率张量（Tensor）的大小。

简单的分解可能会在多个阶段产生这种规模的数组。这并不意味着每个现代框架都同时保持 S和 P处于活动状态，并且编译器融合已经可以避免一些流量。该示例说明了基本问题：密集的 N_2中间体足够大，重复写入和重新读取它可以控制内存使用和带宽。

FlashAttention 避免将完整矩阵（Matrix）存储在 HBM 中。它形成分数图块，在图块位于芯片上时应用 softmax 更新，立即使用这些概率来累积来自V,的相应贡献，然后丢弃该图块。 [4]

> 重要措辞：FlashAttention 无需将完整的注意力机制（Attention）矩阵（Matrix）具体化为片外中间体。它不会消除稠密注意力（Dense Attention）所需的逻辑成对交互。

这也是为什么记忆的好处在训练（Training）期间特别重要，否则朴素的自动梯度会需要大量的中间体来进行向后传递。

5 密集算术仍然是二次的

对应原稿第 9 页

对于具有 N词元（Tokens）和头维度 d的自注意力（Self-attention），形成所有查询（Query）键分数需要成比例的工作

to

$$N^2 d.$$

将概率乘以值会添加另一个相同宽阶的密集成对矩阵乘法（Matrix Multiplication）。 FlashAttention 重新组织了这些操作，但它并没有停止评估密集的查询（Query）键交互集。[4, 5]

所以这三个陈述一定不能混淆：

• 算术复杂度：稠密注意力（Dense Attention）仍然存在 $O(N^2 d)$.

• 大型中间存储：FlashAttention 避免了 HBM 中的 $O(N^2)$物化分数/概率张量（Tensor）。

• HBM 流量：原始论文证明其两级内存模型（Model）下的 IO 成本比标准物化注意力机制（Attention）更低。[4]

这解释了一种方法如何能够在不使长上下文“自由”的情况下使更长的序列变得实用。将 N加倍仍然使密集查询（Query）密钥对的数量大致增加四倍。 FlashAttention 主要攻击该计算的数据移动和内存占用方面。

> 如果您想减少查询（Query）密钥对本身的数量，则需要不同的数学结构：例如稀疏性、局部模式或不同的注意公式。这些选择可以改变模型（Model）行为，并且在概念上与密集 FlashAttention 是分开的。

报告加速时，始终将渐近算术与测量的运行时间区分开来。一个内核（Kernel）可以

在相同的 $O(N^2 d)$复杂度下变得快几倍，因为常数因素、占用、融合和内存流量（Memory Traffic）发生了巨大的变化。

6 内存高效的精确注意力（Exact Attention）早于 FlashAttention

对应原稿第 10 页

从历史上看，说 FlashAttention 首先发现精确注意力（Exact Attention）可以避免二次记忆是不准确的。

Rabe 和 Staats 在 FlashAttention 之前表明，注意力机制（Attention）不需要要求 $O(N^2)$关于序列长度的记忆。他们的工作提供了精确的内存高效算法，同时保留二次时间，以及具有次二次内存的实用加速器实现。 [3]

FlashAttention 的独特贡献是将内存效率转变为显式 IO 感知 GPU 算法：围绕加速器内存层次结构（Memory Hierarchy）进行计算，分析 HBM 访问，融合相关操作，并显示大量的实际运行增益。 [4]

在线softmax也有更早的血统。 Milakov 和 Gimelshein 描述了一种在线递归方法，可以用更少的内存访问来计算经典的稳定 softmax 归一化器。 FlashAttention 将相同类型的 running-max/running-normalizer 想法构建到平铺注意力机制（Attention）中，同时还累积了价值加权输出。 [2, 4]

> 因此，精确的谱系是：• 稳定/在线 softmax 提供流标准化工具，• 早期的内存高效注意力（Efficient Attention）表明精确注意力（Exact Attention）不需要存储 N_2状态，• FlashAttention 共同设计平铺、softmax 状态和 GPU IO 以使该方法在实践中快速。

这种区别很重要，因为“内存效率”和“针对硬件模型（Model）的 IO 最佳”并不是同一说法。

7 平铺查询（Queries）、键和值

对应原稿第 11 页

不要一次形成所有 $N_q N_k$分数，而是将矩阵划分为块。对于一个查询（Query）块 Q_i和一个键/值块 (K_j, V_j),计算

$$S_{ij} = \frac{Q_i K_j^\top}{\sqrt{d}} + B_{ij}.$$

分数tile S_{ij}足够小，可以在计算单元附近进行处理。它的贡献被折叠到运行行统计数据和输出累加器（Accumulator）中，然后可以丢弃该图块。

从概念上讲，对于每个查询（Query）块:

1.加载 Q_i的图块

2.流兼容 K/V图块，

3. 计算一个分数图块，

4.更新行方向的softmax状态，

5.累积对应的 V贡献，

6. 移动到下一个图块，而不写入全局 S或 P矩阵（Matrix）。

实际内核根据片上容量、头维度（Head Dimension）、GPU 生成、因果结构和工作分区来选择瓦片形状和循环顺序。原始 FlashAttention IO 分析使用从 SRAM 容量 M.[4]派生的切片大小

> 仅贴瓷砖是不够的。矩阵乘法（Matrix Multiplication）图块可以自然地组合，因为总和可以累加。Softmax 通过共享的最大值和分母将一行中的每个分数耦合起来，因此我们需要一种方法来合并块，而无需立即看到整行。

这是下一章中的关键数学技巧。

8 Softmax 是流媒体的难点

对应原稿第 12 页

一行的数值稳定 softmax $x_1, \ldots, x_N$使用

$$m = \max_j x_j, \qquad \ell = \sum_j e^{x_j - m}, \qquad p_j = \frac{e^{x_j - m}}{\ell}.$$

m的减法可防止大的正 logits 溢出。但这似乎产生了一个流问题：如果后面的块可能包含更大的最大值，那么如何标准化早期的块？

答案是保留足够的统计数据，以便在最大值发生变化时可以重新调整。假设较早元素之后的运行状态为$(m_{\text{old}}, \ell_{\text{old}})$并且新块的最大值为 m_b.

m_{new}= 最大值(m_{old}, m_b).

在旧的最大值下累积的每个贡献都可以通过乘以转换为新的参考值

it by

$\alpha = e^{m_{\text{old}} - m_{\text{new}}}$.

新块相对于相同的 m_{new}.进行评估

这不是一个近似值。这是身份

$e^{x - m_{\text{old}}} e^{m_{\text{old}} - m_{\text{new}}} = e^{x - m_{\text{new}}}$.

> 运行最大值是数值参考点的变化。当该参考发生变化时，先前累积的指数可以在实际算术中精确地重新缩放，而不是从头开始重新计算。

在线归一化器递归早于 FlashAttention，并为流稳定的 softmax 提供了数学基础。 [2]

9 从第一原理开始的在线softmax

对应原稿第 13 页

一次处理一个标量 logits $x_1, x_2, \ldots$。初始化

$$m_0 = -\infty, \qquad \ell_0 = 0.$$

观察 x_j,更新后

$$m_j = \max(m_{j-1}, x_j),$$

$$\ell_j = \ell_{j-1} e^{m_{j-1} - m_j} + e^{x_j - m_j}.$$

Milakov 和 Gimelshein 表明，这会产生相同稳定的 softmax 归一化器，同时比传统的 safe-softmax 过程需要更少的输入传递。 [2]

为了引起注意，我们还需要加权值总和。引入非标准化累加器（Accumulator）

$$a = \sum_j e^{x_j - m} v_j.$$

当最大值从 m变为 m',时，将 ℓ和 a重新调整为 $e^{m-m'}$.，然后添加新的指数

以及新参考下的价值贡献。

最后，

$$o = \frac{a}{\ell}.$$

For attention, x_j is not a fixed input vector stored in advance. Each block of logits is generated on demand from a matrix product $Q_i K_j^\top / \sqrt{d}$ plus mask/bias terms. The online recurrence lets the kernel 立即消耗该块。

注意， x_j 并不是预先存储的固定输入向量（Vector）。每个 logits 块都是在以下位置生成的

矩阵（Matrix）乘积的需求 $Q_i K_j^\top$

d plus 掩模/偏差项。在线递归让内核（Kernel）

同样的想法逐行、逐块地工作，这使得平铺精确的 softmax-attention 前向传播（Forward Pass）成为可能。

10 分块合并（Blockwise Merge）递归

对应原稿第 14 页

对于一个查询（Query）行，假设一些关键块之后的运行状态是

$$(m, \ell, a),$$

其中 m是迄今为止看到的最大分数， ℓ是该最大值下的稳定softmax分母，$a \in \mathbb{R}^{d_v}$是非归一化值累加器（Accumulator）。

对于具有匹配值 $V_b \in \mathbb{R}^{b \times d_v}$,的新分数块 $s \in \mathbb{R}^b$，让

$$m_b = \max(s), \qquad m' = \max(m, m_b),$$

$$\alpha = e^{m - m'}, \qquad p = e^{s - m'}.$$

然后更新

$$\ell' = \alpha \ell + \sum_r p_r,$$

$$a' = \alpha a + p^\top V_b,$$

$$m \leftarrow m', \quad \ell \leftarrow \ell', \quad a \leftarrow a'.$$

最后，

$$o = a / \ell.$$

对于查询（Query）行块， m和 ℓ成为行向量，a成为矩阵（Matrix）。可以在指数之前将掩码应用于分数图块，掩码位置的概率为零。

> 递归是瓦片边界不会改变密集 softmax 结果的代数原因。不同的平铺会改变浮点（Floating-point）运算的顺序，但不会改变预期的实数算术函数。

FlashAttention 发布的算法以块形式表达等效的 running-max / running-normalizer / 输出更新。 [4, 5]

11 一个完整的数值例子

对应原稿第 15 页

考虑一个已经缩放的、未屏蔽的注意力机制（Attention）行

$$s = [2, 1, 4, 3]$$

具有二维值

$$v_1 = [1, 0],\ v_2 = [0, 1],\ v_3 = [2, 0],\ v_4 = [0, 2].$$

全局最大值为 4，因此稳定的非标准化权重为

$$[e^{-2}, e^{-3}, 1, e^{-1}] \approx [0.135335, 0.049787, 1, 0.367879].$$

他们的总和是

$$\ell \approx 1.55300179,$$

完整的 softmax 给出

$$O \approx [1.37497284,\ 0.50582424].$$

现在处理两个块。对于$[2, 1]$:

$$m_1 = 2, \quad \ell_1 = 1 + e^{-1} = 1.36787944, \quad a_1 = [1, e^{-1}].$$

对于第二个块$[4, 3]$,，新的最大值为 4，因此

$$\alpha = e^{2-4} = e^{-2}.$$

然后

$$\ell_2 = \alpha\ell_1 + 1 + e^{-1} = 1.55300179,$$

$$a_2 = \alpha a_1 + [2, 2e^{-1}] \approx [2.13533528, 0.78554595].$$

因此

$$a_2/\ell_2 \approx [1.37497284, 0.50582424],$$

匹配全行计算。

> 旧街区没有被重新访问。当出现更大的最大值时，它的贡献只是重新调整。

12 FlashAttention 前传

对应原稿第 16 页

一个有用的概念前向传播（Forward Pass）是：

1. 将 Q分区为查询（Query）行图块，将 K, V分区为键/值图块。

2. 对于每个查询（Query）图块，初始化逐行运行最大值、标准化器和输出累加器。

$$^{\top}_{j}/\sqrt{d}.$$

3.加载键/值图块并形成本地分数图块 $Q_i K_j^{\top}$

4. 应用属于此图块的因果/局部掩模或附加偏差。

5. 计算图块最大值，更新运行最大值，并重新调整先前状态。

6. 相对于更新的最大值对当前图块求幂。

7. 更新分母和值加权输出累加器（Accumulator）。

8. 继续，直到所有必需的关键图块都已贡献。

9. 按行标准化累加器（Accumulator）并写入输出。

原始的 FlashAttention 算法选择块大小，以便相关图块和状态适合片上 SRAM，从而减少到 HBM 的行程。[4]

教育实现可以用 PyTorch 的几行代码重现代数，但这样的代码不是高性能（Performance）的 FlashAttention 内核（Kernel）。生产实现取决于 GPU 特定的平铺、线程/线程束（Warp）调度、异步副本、张量（Tensor）核心指令和其他低级细节。

> 基本的算法思想独立于一个 CUDA 内核（Kernel）：生成一个分数图块，通过在线 softmax 和 V累积立即使用它，并且永远不会在 HBM 中具体化完整的分数/概率矩阵（Matrix）。

最低教育 PyTorch

下面的代码 Streams K/V块并直接实现递归。它省略了批处理（Batching）、掩码、丢失、查询（Query）平铺和每个 GPU 优化，因此它是正确性说明而不是 FlashAt-

注意力核。

```
def tiled_attention(q, k, v, block=128):
    scale = 1 / math.sqrt(q.shape[-1])
```

```
m = q.new_full((q.shape[0],), -float("inf"))
l = q.new_zeros(q.shape[0])
a = q.new_zeros((q.shape[0], v.shape[-1]))
```

```
    for j in range(0, k.shape[0], block):
        kj, vj = k[j:j+block], v[j:j+block]
        s = (q @ kj.T) * scale
        m_new = torch.maximum(m, s.max(dim=-1).values)
        alpha = torch.exp(m - m_new)
        p = torch.exp(s - m_new[:, None])
        a = a * alpha[:, None] + p @ vj
        l = l * alpha + p.sum(dim=-1)
        m = m_new
    return a / l[:, None]
```

后来的 FlashAttention 代际保留了这种语义结构，同时改变了在较新硬件上安排工作的方式。[5,6,7]

13 为什么密集的FlashAttention是准确的

对应原稿第 17 页

Dense FlashAttention 被称为精确的，因为目标函数仍然存在

$$\text{softmax}(QK^\top/\sqrt{d}+B)V.$$

平铺不会删除查询（Query）键对。在线 softmax 不会用其他函数代替指数或归一化（Normalization）。块循环只是改变了积累足够统计数据的顺序。[4]

三个注意事项使这个词准确无误：

• 浮点（Floating-point）是有限精度（Precision）的。重排序添加和减少可能会与其他实现产生微小的数值差异。 PyTorch 明确警告 SDPA 后端可能会有所不同，因为浮点运算的融合和顺序不同。[10, 13]

• 训练（Training）过程中，dropout 是随机的。逐位比较两次运行除了数学注意语义之外还需要匹配随机行为。

• 稀疏变体是不同的。原始的 FlashAttention 论文还提出了块稀疏 FlashAttention，它省略了块，因此是相对于完全稠密注意力（Dense Attention）的近似/稀疏变体。 [4]

> “精确”并不意味着“与每个参考内核（Kernel）按位相同”。这意味着该算法没有故意改变密集的softmax注意力机制（Attention）来减少数学工作。

在评估数值测试时，这种区别很重要。合理的容忍度取决于数据类型（Dtype）、累积顺序、序列长度和后端，而不是要求二进制同一性。

14 IO 复杂性：定理的实际含义

对应原稿第 18 页

原FlashAttention论文分析了一个两级内存模型（Model），其中HBM和大小为M.的片上SRAM在论文的假设下，包括头维度 d和

$$d \leq M \leq Nd,$$

标准具体化注意力机制（Attention）需要

$$\Theta(Nd + N^2)$$

HBM 访问，而 FlashAttention 需要

$$\Theta\left(\frac{N^2d^2}{M}\right)$$

$$\Theta\left(\frac{N^2d^2}{M}\right)$$

HBM 访问。该论文还证明了该模型（Model）中一系列 SRAM 尺寸的最优结果。[4]

有几个细节很重要：

• 这些是特定内存模型（Model）中的IO 复杂性结果，而不是每个GPU 的通用字节计数。

• 数量计算标量元素/字在建模内存级别之间的移动，直至渐近因子。

• 这不是算术复杂性。稠密注意力（Dense Attention）仍然执行 $O(N^2d)$工作。

• 增加可用片上存储器 M可实现更多重用并减少建模HBM 流量。

一个常见的错误总结是“FlashAttention 减少了从 $O(N^2)$到 $O(N)$.”的注意力机制（Attention） IO。线性量是相对于序列长度的大辅助内存占用（Memory Footprint），而不是上面一般的 HBM 访问表达式。直观地说，更可用的片上内存可以让更大的工作块驻留（Resident）并在必须重新加载数据之前重用以进行更多注意力机制工作来自 HBM 在定理的体系中，增加的重用是 HBM 访问界限随着 M 的增长而减少的原因。

15 内存复杂度：线性辅助状态，非线性计算

对应原稿第 19 页

为什么FlashAttention论文经常说内存变成线性而不是二次?

输入和输出已包含 $O(Nd)$元素。具体化的稠密注意力（Dense Attention）实现还创建了 $O(N^2)$分数/概率状态。 FlashAttention 避免存储这些完整矩阵，并仅保留平铺工作状态以及按行统计信息。因此，与注意力机制（Attention）操作相关的额外内存随序列长度线性扩展，而不是二次扩展。 [4, 5]

对于训练（Training）来说，这种差异尤其重要，因为直接向后传递可能会保存完整的概率矩阵 P. FlashAttention 而是保存紧凑的信息，例如输出和行归一化（Normalization）统计数据，然后向后重新计算分数/概率图块。 [4, 5]

> 线性内存并不意味着线性运行时间。 N.中密集查询（Query）键交互的数量仍然是二次的

还要避免声称总模型（Model）内存为 $O(N)$.其他 Transformer 组件消耗激活（Activation）内存，并且自回归推理服务（Serving）具有随上下文长度而增长的 KV 缓存内存。 FlashAttention 特别改变了注意力机制（Attention）计算管理其中间体的方式。

这是 FlashAttention 和激活（Activation）检查点可以共存的原因之一：两者都进行重新计算以减少存储状态，但在训练（Training）图的不同范围内。

16 因果掩蔽和瓦片跳跃

对应原稿第 20 页

在因果自注意力（Causal Self-attention）中，查询（Query）位置 i可能不会关注未来的关键位置 $j > i$.物化掩码（Mask）将是另一个 $N \times N$对象，但高效的平铺内核（Kernel）可以推理每个平铺的几何形状。

对于平方自注意力（Self-attention）：

• 严格高于因果边界的块被完全屏蔽并且不需要贡献，

• 严格低于边界的块是完全有效的，

• 只有与对角线相交的块才需要元素级因果掩码。

这避免了在因果内核（Kernel）中计算许多无效的图块。 FlashAttention-2 显式地利用因果结构，而当前的实现还为不相等的查询（Query）/密钥长度定义了精确的对齐规则。 [5, 8]

> 掩码（Mask）语义是 API 的细节，不能被猜测。例如，当前的 PyTorch SDPA 将 Boolean attn_mask 值为 True 视为参与注意力机制（Attention）的位置，而其他 PyTorch mask API 使用不同的约定。 [10]

本地/滑动窗口注意力机制（Attention）同样可以跳过允许窗口之外的图块。但是，一旦模型（Model）有意限制关注哪些对，模型的注意力机制模式就会变得稀疏/局部。 FlashAttention 可以是用于执行该模式的内核（Kernel），但它不再是与无限制稠密注意力（Dense Attention）相同的数学问题。

17 向后传递：重新计算而不是存储

对应原稿第 21 页

一个简单的训练（Training）实现可以保存 $P = \mathrm{softmax}(S)$ 以用于向后。 FlashAttention 避免将 N^2 张量（Tensor）保留在 HBM 中。相反，它存储紧凑的行归一化（Normalization）信息，并在需要梯度时重新计算分数/概率图块。[4, 5]

对于

$$S = QK^\top/\sqrt{d}, \qquad P = \mathrm{softmax}(S), \qquad O = PV,$$

一个有用的行式恒等式是

$$D_i = \sum_r (dO_{ir})O_{ir}.$$

重新计算 P 的图块后，局部导数可以表示为

$$dV \mathrel{+}= P^\top dO,$$

$$dP = dO\, V^\top,$$

$dS = P \odot (dP - D[:, \text{无}])$,

然后

$$dQ \mathrel{+}= dS\, K/\sqrt{d}, \qquad dK \mathrel{+}= dS^\top Q/\sqrt{d}.$$

掩码意味着掩码条目的概率/梯度（Gradient）贡献为零。

FA2 存储逐行 log-sum-exp 数量，允许根据重新计算的分数稳定地重建概率图块。 [5]

> 向后重新计算是有意的。它花费额外的矩阵（Matrix）乘法工作来避免读取和写入巨大的概率张量（Tensor），这对 GPU 来说是一个有利的交易。

18 为什么更多的 FLOP 仍然可以更快

对应原稿第 22 页

人们很容易认为更少的算术运算总是意味着更低的延迟（Latency）。加速器的性能（Performance）经常打破这种直觉。

矩阵乘法（Matrix Multiplication）与张量核的映射非常好。 HBM 流量、同步（Synchronization）、共享内存流量、指数、减少和内核启动（Kernel Launch）边界可能相对昂贵。因此，FlashAttention 会进行一些交易，例如向后重新计算图块或进行重新缩放算术（如果这会阻止更大的内存传输）。[4, 5]

FA2 明确了这一对比：其目标之一是减少非 matmul FLOP，因为这些操作不享有与张量（Tensor）核心 GEMM 相同的吞吐量（Throughput）。该论文还改进了工作分区，因此占用了更多的 GPU。[5]

FA3 在 Hopper 的基础上更进一步，通过使用异步硬件功能来重叠矩阵乘法（Matrix Multiplication）、softmax 和数据移动（Data Movement）。 FA4 响应 Blackwell，其中张量核心吞吐量（Tensor-core Throughput）比其他一些资源增长得更快，使得指数和共享内存流量（Memory Traffic）相对更加重要。[6, 7]

> 更好的性能（Performance）问题不仅仅是“多少次 FLOP？”而是“哪些操作、哪些单元、哪些数据移动（Data Movement）、重用、并行性和同步（Synchronization）？”

这是使 FlashAttention 变得比注意力机制（Attention）本身更重要的系统课程：硬件效率通常来自于与内存/执行层次结构的共同设计数学调度。

19 MHA、MQA 和 GQA 兼容性

对应原稿第 23 页

FlashAttention 并不要求每个架构都具有相同数量的查询（Query）和键/值头（Value Heads）。

让

$$Q \in \mathbb{R}^{B\times N_q\times H_q\times d}, \qquad K,V \in \mathbb{R}^{B\times N_k\times H_{kv}\times d}.$$

对于普通的 MHA, $H_q = H_{kv}$.在 MQA, $H_{kv} = 1$ 中。在 GQA,$1 < H_{kv} < H_q$.中，当前 Dao-AILab 内核通过传递比查询头（Query Heads）更少的 KV 头来支持 MQA/GQA，要求查询头的数量可以被KV头的数量整除。[8]

内核（Kernel）仍然评估每个查询头（Query Head）与其分配的 KV 头之间的注意力机制（Attention）。头部共享改变了架构和 KV 内存占用（Memory Footprint）。 FlashAttention 改变了结果注意操作的执行方式。

> 正交概念： • MQA/GQA：头部如何共享 K/V 投影， • RoPE：位置如何转换 Q/K， • FlashAttention：如何有效地安排和计算注意力机制（Attention）。这些可以一起使用。

当前的 PyTorch SDPA 还公开了enable_gqa。其文档标签 GQA 支持实验性并施加后端和张量（Tensor）形状约束，因此生产代码应遵循确切版本的文档，而不是假设通用融合内核（Kernel）支持。 [10]

20 可变长度、局部注意力（Local Attention）和 dropout

对应原稿第 24 页

生产注意力机制（Attention）很少只包含没有丢失的等长密集序列。当前的 FlashAttention 实现支持更广泛的功能集，但这些是实现功能，而不是数学思想本身的属性。

当前的 Dao-AILab 存储库记录了功能的内核/接口，包括：

• 可变长度序列，

• 因果关注，

• 本地/滑动窗口注意力机制（Attention），

• 面向培训的界面中的退出，

• MQA/GQA，

• 相关界面中的 ALiBi 式评分偏差，

• 具有键值缓存（KV Cache）的专用增量解码路径，包括可选的RoPE 处理。[8]

功能支持因 CUDA/ROCm 后端而异，并随着时间的推移而发展。例如，存储库记录了具有不同实现细节和支持矩阵的单独 NVIDIA 和 AMD 后端。 [8]

辍学者应该受到实际的警告。当前的 PyTorch scaled_dot_product_attention 始终根据其 dropout_p 参数应用 dropout，因此当应禁用 dropout 时，调用者必须在评估期间通过0.0。 [10]

> 请勿从名称“FlashAttention”推断功能支持。检查实际运行的确切库、内核（Kernel）生成（Generation）、设备、数据类型（Dtype）、头维度（Head Dimension）、掩码（Mask）/偏差以及训练（Training）/推理（Inference）路径。

21 FlashAttention-2：发生了什么变化

对应原稿第 25 页

FlashAttention-2没有改变根本目标。它仍然计算精确的注意力，同时避免二次物化中间体。改进主要是为了更好地使用 GPU。[5]

FA2 论文确定了三个主要变化：

1.减少非matmul FLOP的数量，

2. 即使对于单个头，也可以跨序列图块并行注意力机制（Attention），以提高占用率（Occupancy），

3. 在线程块（Thread Block）内的 warp 之间重新分配工作，以减少共享内存通信。[5]

在本文的 A100 实验中，这些变化比 FlashAttention-1 产生了大约2×的加速，并达到理论最大 FLOPs/s 的 50-73%，而在报告的设置下，端到端 GPT 式训练（Training）达到每 A100 225 TFLOPs/s。 [5]

这些数字并不是普遍的速度保证。它们取决于 GPU、形状、数据类型（Dtype）、因果模式、软件版本和基准测试（Benchmark）方法。

> FA1 的突破在于 IO 感知平铺精确注意力（Exact Attention）。 FA2 的突破在于更好的并行性和围绕相同高级算法思想的工作划分（Work Partitioning）。

这种模式将重复：后代保留语义，但使内核（Kernel）管道适应较新的 GPU 架构的资源和瓶颈。

22 FA2 跨序列图块的并行性

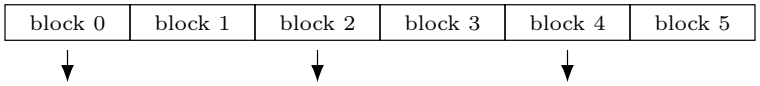
对应原稿第 26 页

GPU 需要足够的独立工作来保持许多流式多处理器繁忙。在 FA1 中，并行性的自然单位大致是 batch × head。如果批大小（Batch Size）和人数都很小，即使序列本身很长，也可能导致硬件利用率不足。

FA2 沿序列维度添加并行性。对于相同的注意力头（Attention Head），不同的线程块可以在不同的查询（Query）行图块上工作。因为查询行在它们自己的键/值减少完成之前是独立的，所以这在不改变注意语义的情况下公开了更多并行工作。 [5]

一个简化的心理图景是：

Q 行



区块0

区块1

区块2

区块3

区块 4

区块5

CTA CTA CTA

重要的一点不是确切的块分配，这是特定于内核（Kernel）的。正是长序列长度本身成为并行性的来源。

> 并行块越多并不一定越好。平铺大小、占用、寄存器（Registers）、共享内存（Shared Memory）和数据重用相互作用。内核（Kernel）设计是对硬件资源的优化问题，而不仅仅是最大线程数的要求。

23 FA2 工作分区和非 matmul FLOP

对应原稿第 27 页

FA2 还改变了每个线程块（Thread Block）内的工作。 FA1 的线程束（Warps）分区可能需要将部分结果写入共享内存（Shared Memory）并进行组合，从而产生通信开销。 FA2 重新分区工作，因此线程束以减少共享内存读/写的方式进行协作。[5]

该论文还减少了相对于张量（Tensor）核心矩阵乘法（Matrix Multiplication）而言昂贵的运算。这包括围绕 softmax 重新缩放和向后计算进行代数重组，从而减少执行的非 matmul FLOP。 [5]

这很重要，因为“一次 FLOP”并不是适用于所有操作类型的有用性能（Performance）货币。张量核可以以巨大的吞吐量（Throughput）执行矩阵（Matrix）乘法累加，而指数、标量缩减、转换和共享内存同步（Synchronization）则有不同的限制。

> FA2 提醒的是，IO 改进后，下一个瓶颈可能是占用、warp 通信或非 matmul 工作。优化是移动瓶颈，而不是使硬件限制消失。

FA2 还细化了向后保存的内容，使用紧凑的行对数和表达式信息并重新计算图块而不是具体化概率。这保留了线性内存特性，同时改善了内核（Kernel）的工作分配。[5]

24 FlashAttention-3：Hopper一代

对应原稿第 28 页

Hopper GPU 引入的硬件功能改变了安排注意力机制（Attention）的最佳方式。 FlashAttention-3 是专门为利用这些功能而设计的。[6]

FA3 论文指出，FA2 在测量中仅在 H100 上实现了约 35% 的利用率，留下了巨大的空间。它提出了三个广泛的想法：

1. 使用异步和线程束（Warp）专门化进行重叠计算和数据移动（Data Movement），

2.交错矩阵乘法（Matrix Multiplication）和softmax工作，使不同的执行单元保持忙碌，

3. 利用 FP8 硬件和旨在控制数值误差的量化（Quantization）技术。 [6]

> 在 H100 上，NeurIPS 2024 最终出版物报告其基准套件中的速度比 FA2 提高了 1.5-2.0 倍，BF16 吞吐量（Throughput）高达 840 TFLOPs/s（85% 利用率），FP8 吞吐量达到 1.3 PFLOPs/s。 [6]

同样，这些是特定于硬件和基准的经验结果，而不是对任意模型（Models）的承诺。

> FA3 并不是“更近似的 FlashAttention”。 FP16/BF16 路径保留了精确注意算法目标。 FP8 路径有意引入较低精度（Precision）的算术，因此需要对其自己的数值精度进行讨论。

25 FA3 异步：重叠数据移动（Data Movement）、GEMM 和 softmax

对应原稿第 29 页

在 Hopper 上，张量内存加速器（Tensor Memory Accelerator）（TMA）和异步线程束（Warps）组矩阵（Matrix）乘法运算使得构建更深层次的软件管道成为可能。FA3 使用 warp 专门化，因此不同的 warp 负责不同的阶段，从而允许数据移动（Data Movement）和计算重叠。[6]

简化的管道是：

$$\| \quad QK^\top \text{ GEMM} \quad \| \quad \text{softmax/update} \quad \|$$

$QK^\top$GEMM

加载下一个图块

Softmax/更新

PVGEMM.

符号 ‖表示各个阶段在依赖性允许的情况下有意重叠，而不是说它们在数学上是独立的。

FA3 还使用乒乓式调度来交错块矩阵乘法（Matrix Multiplication）和 softmax。重点是避免在执行标量/特殊功能工作时让张量核心（Tensor Cores）闲置，反之亦然。[6]

> online-softmax 递归在概念上仍然存在。变化的是用于执行评分、标准化和值累积阶段的硬件管道。

这是该系列中的一个重要进展：FA1 询问“我们如何避免 HBM 流量？”，FA2 询问“我们如何更好地划分工作？”，FA3 询问“我们如何在 Hopper 上重叠不同的执行资源？”

26 FA3 FP8：不假装精确的性能（Performance）是免费的

对应原稿第 30 页

Hopper 为 FP8 提供非常高的张量核心吞吐量（Tensor-core Throughput）。然而，简单地将所有注意力机制（Attention）操作数转换为 FP8 可能会产生不可接受的数值错误，因为注意力机制结合了点积、指数、归一化（Normalization）和随图块变化的动态范围内的值累积。

> FA3 引入了使用块量化（Quantization）和非相干处理技术的 FP8 路径，以减少量化误差（Quantization Error），同时利用 FP8 张量核心吞吐量（Tensor-core Throughput）。在 NeurIPS 2024 最终出版物中，FP8 FA3 在 H100 上达到 1.3 PFLOPs/s，并且报告的数值误差比用于比较的基线（Baseline） FP8 注意力机制（Attention）方法低 2.6 倍。 [6]不要混淆“精确”的两个含义。FlashAttention 的平铺/在线 softmax 算法对于稠密注意力（Dense Attention）公式来说可以是精确的，而以较低精度（Precision）的数字格式执行该公式仍然会引入量化/舍入（Rounding）误差。

模型（Model）可接受的精度（Precision）取决于完整的训练（Training）/推理（Inference）设置。因此，FP8 支持是分层到 FlashAttention 执行策略上的硬件和数字功能，而不是证明 FP8 可以与 BF16/FP16 普遍互换。

官方存储库继续将 FA3 的 Hopper 特定实现与其他后端分开，这是不将 FA3 功能声明为“FlashAttention”通用属性的另一个原因。[8]

27 FlashAttention-4：布莱克威尔一代

对应原稿第 31 页

FlashAttention-4 延续了相同的硬件感知重新设计模式，这次是针对 NVIDIA Blackwell。该论文认为 Blackwell 表现出不对称的硬件扩展：张量核心吞吐量（Tensor-core Throughput）相对于 Hopper 大约增加了一倍，而其他一些资源，包括共享内存带宽（Memory Bandwidth）和指数吞吐量（Throughput），并没有以相同的速率扩展。 [7]

当一个子系统变得更快时，曾经次要的操作可能会成为瓶颈。

因此，FA4 重新设计了前向和后向管道，而不仅仅是重用 Hopper 调度表。

论文的主要技术包括：

• 完全异步MMA管道和更大的图块，

• 软件模拟指数加条件在线softmax 前向重新缩放，

• 张量（Tensor）内存和 2-CTA MMA 技术可减少向后共享内存流量（Memory Traffic）和原子添加。[7]

在使用 BF16 的 B200 上，该论文报告说，与 cuDNN 9.13 相比，加速高达1.3×，与 Triton 比较相比，2.7×加速，在评估的配置下达到 1613 TFLOPs/s（71% 利用率）。 [7]

> FA4 并没有改变稠密注意力（Dense Attention）的大 O 算法。它攻击了 GPU 一代所产生的新瓶颈，其计算/内存/功能单元平衡与 Hopper 不同。

28 FA4 和非对称硬件缩放（Scaling）

对应原稿第 32 页

FlashAttention 谱系是一个案例研究，说明为什么内核不能一次优化并假设永远最佳。

在 Blackwell 上，矩阵乘法（Matrix Multiplication）变得足够快，以至于前向传播（Forward Pass）可能会受到相对于张量（Tensor）核心 GEMM 的 softmax/指数工作的限制。 FA4 使用软件模拟指数，并在运行最大值不需要时避免不必要的重新缩放，从而减轻较慢的非 matmul 资源的压力。 [7]

向后传递有一个不同的瓶颈。 FA4 使用 Blackwell 的张量（Tensor）内存 (TMEM) 和 2-CTA MMA 模式来减少共享内存流量（Memory Traffic）和原子累积开销。 [7]

这就产生了一个重要的系统原理：

> 内核（Kernel）根据特定的硬件比率进行平衡。如果张量核心吞吐量（Tensor-core Throughput）的提高速度快于内存带宽（Memory Bandwidth）或特殊函数吞吐量（Throughput），则即使数学函数相同，最佳算法管道也会发生变化。

这就是为什么“FlashAttention-4 只是更快的 FA3”错过了有趣的部分。该算法和内核（Kernel）管道是围绕新的不对称性共同设计的。 FlashAttention-4 标题本身就明确表明了这一点。[7]

这也解释了为什么性能（Performance）数字应始终命名为 GPU 一代。 B200 上的加速并不证明 H100、A100 或非 NVIDIA 加速器上具有相同的比率。

29 FA4实施及现状

对应原稿第 33 页

当前的官方 Dao-AILab 存储库将 FlashAttention-4 公开为针对 Hopper 和 Blackwell GPU（例如 H100 和 B200）进行优化的 CuTeDSL 实现，并通过 flash-attn-4 软件包进行安装。 [8]

FlashAttention-4 论文强调了另一个工程变化：实现是用嵌入 Python 的 CuTe-DSL 编写的。该论文报告说，20–30×的编译时间比比较中使用的传统 C++ 模板方法更快，同时保留了所需的低级表达能力。 [7]

> 截至 2026 年 9 月，PyPI 将 flash-attn-4 分类为开发状态 :: 3 - Alpha。[9]将其视为当前实现快照，而不是常青属性。“FA4 存在于官方存储库中”和“每个生产环境都应该用 FA4 替换 FA2”是截然不同的说法。

硬件、CUDA、PyTorch、功能和封装要求快速发展。持久的教训是算法进展，而在部署之前应始终根据当前的官方存储库检查确切的安装/支持声明。

30 FlashAttention-1 到 -4 比较

对应原稿第 34 页

	主要目标	核心内核/算法侧重点	选定的论文报告结果
FA1	IO/HBM 流量	平铺、在线softmax、融合精确注意力	NeurIPS 2022 论文报告了实质性内容

化，IO分析

与当前基线相比的初步训练（Training）加速。[4]

FA2

GPU 利用率（GPU Utilization）更多的序列级并行性、更少的非 matmul FLOP、改进的线程束（Warp）工作分区

关于论文中 FA1 上的2×和 A100 上 50-73% 的理论最大 FLOPs/s。[5]

纸质 H100 上 FA2 的 1.5-2.0 倍，
BF16 高达 840 TFLOPs/s。[6]

FA3

Hopper Warp 专业化，异步 TMA/WGMMA 管道，GEMM-softmax 重叠，FP8 路径

FA4

布莱克威尔时代的不对称性

完全异步 MMA 管道、更大的图块、软件 exp/条件重新缩放、TMEM 和 2-CTA 向后技术

B200 上的 BF16 和1.3×与论文中的 cuDNN 9.13 相比高达 1613 TFLOPs/s。[7]

> 版本号不应被解释为不同注意力机制（Attention）定义的序列。该系列保留了高效注意力（Efficient Attention）评估的中心目标，同时调整调度和数值路径以适应更新的硬件。

确切的特征矩阵（Matrix）因实现而异。例如，FA3 与 Hopper 特定的优化密切相关，而当前的 FA4 存储库通过其 CuTeDSL 路径同时针对 Hopper 和 Blackwell。 [8]

31 PyTorch 缩放点积今日关注

对应原稿第 35 页

高级 PyTorch API 是

```
torch.nn.functional.scaled_dot_product_attention(
    query, key, value,
    attn_mask=None,
    dropout_p=0.0,
    is_causal=False,
    scale=None,
    enable_gqa=False,
)
```

Current PyTorch documentation says CUDA SDPA can automatically select optimized implementations based on the inputs. The main API documentation explicitly discusses FlashAttention-2, a memory-efficient attention implementation, and the C++ math implementation. The preferred mechanism for restricting the backend is `torch.nn.attention.sdpa_kernel`.[10, 11]

例如：

```
from torch.nn.attention import SDPBackend, sdpa_kernel
import torch.nn.functional as F

with sdpa_kernel(SDPBackend.FLASH_ATTENTION):
    y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
```

当前的 PyTorch 文档称 CUDA SDPA 可以根据输入自动选择优化的实现。主要的 API 文档明确讨论了 FlashAttention-2、内存高效的注意力机制（Attention）实现和 C++ 数学实现。限制后端的首选机制是 torch.nn.attention.sdpa_kernel.[10, 11]

如果请求的融合内核（Fused Kernel）无法针对提供的输入运行，PyTorch 可以在禁用其他回退时发出警告并说明原因。后端资格取决于设备、数据类型（Dtype）、形状、掩码（Mask）和其他参数。[10]

当前的 torch.nn.attention 文档还公开了较新的 FlashAttention 实现的注册/激活（Activation）挂钩，包括 FA3 和 FA4 等标识符。这是一个快速移动的集成表面，因此请使用已安装的 PyTorch 版本的文档作为权威。[12]

32 精确性不是按位恒等

对应原稿第 36 页

假设参考数学内核（Kernel）和 FlashAttention 内核都实现相同的密集 SDPA 定义。您仍然应该预料到较小的数值差异。

浮点（Floating-point）加法不具有关联性：

$$(a+b)+c \neq a+(b+c)$$

某些值的精度（Precision）有限。平铺改变了缩减顺序。当值四舍五入时，融合可能会发生变化。累积精度可能有所不同。 PyTorch 的再现性文档明确指出，SDPA 后端可以产生不同的结果，因为它们以不同的顺序执行浮点（Floating-point）累加（Accumulation）。 [13]

因此，数值验证应使用适当的容差而不是按位相等。预期的容差取决于：

- FP32 vs. BF16/FP16/FP8,

• 累积精度（Precision），

• 序列长度和分数分布，

• 硬件/后端，

• 丢失/随机性。

> 精确的算法意味着没有刻意的稠密注意力（Dense Attention）的数学近似。按位相同的实现意味着每个有限精度（Precision）运算都位于完全相同的位上。 FlashAttention 承诺第一个想法，而不是跨任意内核的第二个想法。

决定论是另一个单独的问题。当前的库公开了后端和特定于版本的确定性行为控制，这可能会带来性能（Performance）或内存成本。 [8, 13]

33 FlashAttention 与 PagedAttention

对应原稿第 37 页

这些名称听起来相关，但它们解决不同的问题。

	FlashAttention	分页注意力
主要问题	有效地执行注意力机制，尤其是	管理动态增长的 KV 缓存
减少大量中间内存流量（Memory Traffic）		
LLM 推理服务（Serving）的内存		
机制平铺、在线 softmax、融合/硬件感知内核		
受分页启发的 KV 块分配和映射 主状态 注意图块和行归一化（Normalization）/输出状态		
每个请求的持久 键值缓存（KV Cache）		
改变稠密注意力		
否，对于密集 FlashAttention 否，主要更改缓存内存管理		
tion formula?		agement

PagedAttention was introduced with vLLM to reduce KV-cache waste from fragmentation and duplication

化公式?

分页注意力（PagedAttention）与 vLLM 一同提出，用于减少碎片化（Fragmentation）和重复存储造成的 KV 缓存浪费

代理

在高通量推理服务（Serving）中。[14]

FlashAttention是关于注意力机制（Attention）操作如何消耗Q/K/V的。 PagedAttention 是关于推理服务（Serving）系统如何跨请求布局和访问持久 KV 状态。它们可以共存于同一个推理服务堆栈中。

当前的 Dao-AILab 存储库甚至公开了一个面向 KV 缓存的 FlashAttention 接口以及可选的块表，说明内核（Kernel）执行和缓存分页是可组合的问题，而不是相互排斥的想法。 [8]

> 如果问题陈述包含“N_2分数/概率中间值”，请考虑 FlashAttention。如果它包含“分段的每个请求 KV 缓存分配”，请考虑 PagedAttention。

34 FlashAttention 与稀疏和线性注意力（Linear Attention）

对应原稿第 38 页

三种截然不同的策略通常被组合在一起，因为在某些情况下，所有策略都可以降低注意力机制（Attention）成本。

Dense FlashAttention 保留了密集的 softmax-attention 函数并改变了它的执行方式。

稀疏/局部注意力（Local Attention）有意仅计算查询（Query）键交互的子集。如果每个查询仅关注一个窗口或选定的块，则算术可能会低于完全密集 N^2的工作，但模型（Model）的注意力机制（Attention）模式已经改变。

线性注意力机制（Attention）族改变了数学公式，通常使用特征图或关联性，因此可以在没有显式密集 softmax 得分矩阵的情况下重新排列注意力机制。它们的语义和近似/精确性属性取决于具体的方法。

> “记忆高效注意力（Efficient Attention）”这个词太宽泛，无法确定一种方法。问两个问题： 1. 它是否计算相同的密集 softmax 注意力机制（Attention）？ 2. 它是否减少算术对，或主要减少内存流量（Memory Traffic）/中间存储？

原始的 FlashAttention 论文明确地将其密集精确算法与近似方法进行了对比，并单独探索了块稀疏 FlashAttention 作为近似/稀疏扩展。 [4]

这就是为什么“FlashAttention 使注意力机制（Attention）呈线性”是错误的。如果系统由于使用局部窗口（Local Window）或其他稀疏模式而显示出近线性缩放（Scaling），则稀疏性会改变交互数量。FlashAttention 可能仍然是底层的内核（Kernel）。

35 训练（Training）、预填充（Prefill）和解码（Decode）是不同的机制

对应原稿第 39 页

当查询（Query）和关键序列维度都很大时，例如在训练（Training）或长提示预填充（Prefill）中，FlashAttention 自然是最强大的。有很多查询行，因此避免物化注意力机制（Attention）矩阵（Matrix）并利用平铺 GEMM 会带来很大的好处。

自回归解码（Decode）则不同。对于单个新词元（Token），N_q可能是 1，而 N_k是累积的上下文长度。该操作读取大量 键值缓存（KV Cache），但每个序列仅生成一个新查询（Query）行。在这种情况下，键值缓存带宽和推理服务（Serving）/批处理（Batching）行为可能占主导地位，并且专门的解码内核（Kernel）很重要。官方存储库公开了 flash_attn_with_kvcache，它可以更新和处理一个内核中的缓存，并支持 MQA/GQA 和可选 RoPE 处理等功能。 [8]

> 这并不意味着 FlashAttention 与推理（Inference）无关。预填充（Prefill）是一种高度关注的密集机制，解码（Decode）仍然可以使用同一实现系列中的专用内核。这意味着“FlashAttention 加速 LLM 推理”是不完整的，除非你说出哪个阶段和瓶颈。

FlashAttention 也无法消除其他成本：MLP 层、张量（Tensor）并行通信、KV 缓存容量（Cache Capacity）、模型（Model）权重带宽、采样和调度程序开销仍然是单独的问题。

36 种常见的实施错误

对应原稿第 40 页

1. 假设 flash 后端始终运行。当数据类型（Dtype）、设备、形状或掩码（Mask）不受支持时，高级框架可能会回退。在重要的时候使用后端控制/分析。[10]

2. 在评估期间保留辍学。 PyTorch SDPA 完全按照提供的方式应用 dropout_p，因此当评估不需要 dropout 时通过0.0。[10]

3. 使用错误的掩码（Mask）约定。布尔掩码（Boolean Mask）语义因 API 而异。

4. 期望按位一致。融合内核可以重排序浮点（Floating-point）运算。[13]

5. 忽略GQA 头约束。当前接口需要兼容/可分割的查询（Query）和 KV 头数。[8, 10]

6. 将当前的头维度/dtype 支持视为永恒的。这些限制会随着版本和后端的不同而变化。[8]

7. 无需同步（Synchronization）/预热的基准测试。异步 GPU 执行可能会使简单的实际运行测量变得毫无意义。

8. 将稀疏/局部模式称为“密集 FlashAttention”。内核（Kernel）和注意力机制（Attention）模式是单独的选择。

9. 混淆预填充（Prefill）和解码（Decode）。形状和瓶颈不同。

10. 假设最新一代始终是可部署的。当前 FA4 包元数据是 alpha，必须检查硬件/软件兼容性。 [9]

37 常见误解和实用心智模式

对应原稿第 41 页

“FlashAttention 近似于 softmax。”

否。Dense FlashAttention 使用平铺在线归一化（Normalization）来评估密集的 softmax-attention 函数。[4]

“FlashAttention 引起关注 $O(N)$.”

不会。密集算术仍然是 $O(N^2d)$.大的辅助注意力机制（Attention）记忆占用空间在序列长度上变得线性，并且HBM 流量减少。

“这只是内核（Kernel）融合。”

Fusion 是实现过程的一部分，但在线 softmax 平铺和 IO 感知调度是算法性的，而不仅仅是连接现有内核。

“它与 PagedAttention 相同。”

不会。PagedAttention 管理用于推理服务（Serving）的 KV 缓存分配。[14]

“精确意味着按位相同。”

不可以。浮点（Floating-point）运算顺序可以更改舍入（Rounding）。[13]

“长上下文变得自由。”

不会。密集成对算术仍然是二次的，并且 KV 缓存/其他模型（Model）成本仍然存在。

“FA1、FA2、FA3 和 FA4 是不同的注意力机制（Attention）架构。”

不。它们是由不同硬件瓶颈塑造的连续几代高效内核/算法。[4,

$$5, 6, 7]$$

实用理解框架（Mental Model）：FlashAttention 将分数图块保持在靠近计算单元的位置，仅携带精确合并 softmax 块所需的行统计信息，将每个概率图块立即消耗到 V累积中，并避免通过 HBM 往返完整的 N_2注意力机制（Attention）矩阵（Matrix）。

参考文献和进一步阅读

对应原稿第 42 页

参考文献

[1] Vaswani, A. et al. 您所需要的就是注意力机制（Attention）。 NeurIPS 2017.https://arxiv.org/abs/1706.03762

原题名与书目信息：[1] Vaswani, A. et al. Attention Is All You Need. NeurIPS 2017. https://arxiv.org/abs/1706.03762

[2] Milakov, M., Gimelshein, N.softmax 的在线标准化器计算。 arXiv:1805.02867, 2018.https://arxiv.

原题名与书目信息：[2] Milakov, M., Gimelshein, N. Online normalizer calculation for softmax. arXiv:1805.02867, 2018. https://arxiv.org/abs/1805.02867

[3] Rabe, M. N., Staats, C. 自注意力（Self-attention）不需要$O(n^2)$内存。 arXiv:2112.05682，2021/2022。https：

原题名与书目信息：[3] Rabe, M. N., Staats, C. Self-attention Does Not Need $O(n^2)$ Memory. arXiv:2112.05682, 2021/2022. https://arxiv.org/abs/2112.05682

[4] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C. FlashAttention：快速且内存高效的精确注意力（Exact Attention）

原题名与书目信息：[4] Dao, T., Fu, D. Y., Ermon, S., Rudra, A., Ré, C. FlashAttention: Fast and Memory-Efficient Exact Attention 具有 IO 意识。 NeurIPS 2022.https://arxiv.org/abs/2205.14135

[5] Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR 2024。https：

原题名与书目信息：[5] Dao, T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. ICLR 2024. https://arxiv.org/abs/2307.08691

[6] Shah, J., Bikshandi, G., Zhang, Y., Thakkar, V., Ramani, P., Dao, T. FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision. NeurIPS 2024。DOI：10.52202/079017-2193。

[7] Zadouri, T.、Hoehnerbach, M.、Shah, J.、Liu, T.、Thakkar, V.、Dao, T. FlashAttention-4：算法和内核（Kernel）

原题名与书目信息：[7] Zadouri, T., Hoehnerbach, M., Shah, J., Liu, T., Thakkar, V., Dao, T. FlashAttention-4: Algorithm and Kernel 非对称硬件扩展的流水线协同设计。 arXiv:2603.05451, 2026.https://arxiv.org/abs/2603.05451

[8]道-AILab。 flash-attention 官方存储库和当前实施文档。 https://github.com/

原题名与书目信息：[8] Dao-AILab. flash-attention official repository and current implementation documentation. https://github.com/

Dao-AILab/flash-attention

[9]PyPI。 flash-attn-4 包元数据和发布历史记录；包分类器“开发状态::3 - Alpha”。 https://pypi.org/project/flash-attn-4/

[10] PyTorch。 torch.nn.function.scaled_dot_product_attention 文档。 https://docs.pytorch.org/docs/

原题名与书目信息：[10] PyTorch. torch.nn.functional.scaled_dot_product_attention documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html

[11] PyTorch。 torch.nn.attention.sdpa_kernel 文档。 https://docs.pytorch.org/docs/stable/generated/

原题名与书目信息：[11] PyTorch. torch.nn.attention.sdpa_kernel documentation. https://docs.pytorch.org/docs/stable/generated/torch.nn.attention.sdpa_kernel.html

[12] PyTorch。 torch.nn.attention 模块和 FlashAttention 实现注册 API。 https://docs.

原题名与书目信息：[12] PyTorch. torch.nn.attention module and FlashAttention implementation registration APIs. https://docs.pytorch.org/docs/stable/nn.attention.html

[13] PyTorch. 再现性文档，包括缩放点积注意后端差异。 https：

原题名与书目信息：[13] PyTorch. Reproducibility documentation, including scaled-dot-product attention backend differences. https://docs.pytorch.org/docs/stable/notes/randomness.html

[14] Kwon, W. et al. 使用 PagedAttention 进行大型语言模型的高效内存管理。 SOSP

原题名与书目信息：[14] Kwon, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023.https://arxiv.org/abs/2309.06180

参考图：针对哪个问题阅读什么内容

对应原稿第 43 页

问题	最佳起始来源
原来的IO是什么-	FlashAttention-1论文[4]

知道算法吗？

为什么在线标准化有效？

米拉科夫和吉梅尔谢因 [2]

确切的次二次记忆注意力机制（Attention）是否更早已知？

拉贝与斯塔茨 [3]

具体改变了什么

FlashAttention-2 论文 [5]

in FA2?

Hopper 的具体内容是什么

FlashAttention-3 论文 [6]

FA3?

布莱克威尔发生了什么变化

FlashAttention-4 论文 [7]

in FA4?

目前官方包支持什么？

Dao-AILab 存储库 [8]

当前PyTorch如何选择SDPA内核？

PyTorch SDPA 和 sdpa_kernel 文档 [10, 11]

为什么确切的后端在数字上会有所不同？

PyTorch 再现性注释 [13]

PagedAttention 有何不同？

权等人。 [14]

> 实施支持的变化速度比数学还要快。对于已部署的系统，将已安装框架的文档和确切的内核（Kernel）存储库/版本视为受支持的设备、数据类型、头维度、掩码、丢失和 GQA 行为的权威。

面向人工智能工程师、开发人员和认真学习者的简短技术手册